OCOM5300M Artificial Intelligence Project ROCOv2 Encoder

In [1]:
```
# Custom Classes
from images import Images
from pipeline import Encoder

# Deep learning Functionality
import torch

# Image Processing And Computer Vision Functionality
from torchvision import transforms

# Data Handling And Analysis
import pandas as pd

# Visualization
import matplotlib.pyplot as plt # plotting and visualizing results

# Utilities
import os
```

In [2]:
```
# suppress Intel specific Math Kernel Library (MKL) warnings
os.environ['KMP_DUPLICATE_LIB_OK'] = 'True'
```

In [3]:
```
# CPU
device = torch.device('cpu')
```

In [4]:
```
# display the complete content of DataFrame columns without truncation
pd.set_option('display.max_colwidth', None);
```

Processing Pipeline

The following transform will be used :

In [5]:
```
# image processing pipeline
transform = transforms.Compose([
```

```
    transforms.Resize(
        (224, 224), # consistent with RadImageNet ResNet-50 input and ROCOv2
        antialias=True, # to prevent jagged edges, moire patterns etc
    ),

    # PyTorch neural network requires the image as a tensor of numerical values
    transforms.ToTensor(), # scales pixel values from 0-255 to 0-1

    # convert RGB tensor to BGR to match RadImageNet PyTorch preprocessing
    transforms.Lambda(lambda image: image[[2, 1, 0], :, :]),

    # maps values from 0-1 to -1 to 1, matching the intensity scaling used in the RadImageNet preprocessing
    transforms.Normalize(
        mean=[0.5, 0.5, 0.5],
        std=[0.5, 0.5, 0.5],
    ),
])
```

In [6]:
```
# instantiate encoder
encoder = Encoder().to(device)
```

Extract Training Set Features

In [7]:
```
# relative path to ROCOv2 training images
train_image_folder = 'ROCOv2/train_images/train'
```

In [8]:
```
# instantiate Images object
train_images = Images(train_image_folder, '.jpg', transform)
```

In [9]:
```
# extract image features
train_images.extract_features(encoder, device);
```

Extract Validation Set Features

In [10]:
```
# relative path to ROCOv2 validation images
valid_image_folder = "ROCOv2/valid_images/valid"
```

In [11]:
```
# instantiate Images object
valid_images = Images(valid_image_folder, '.jpg', transform)
```

In [12]:
```
# extract image features
valid_images.extract_features(encoder, device);
```

Diagnostics

In [13]:
```
def analyse_feature_distribution(train_image_folder, valid_image_folder, maximum_files=None):

    """
    Calculate and plot feature-distribution statistics for the training and validation datasets

    Args:
        train_image_folder (str): training image folder
        valid_image_folder (str): validation image folder
        maximum_files (int or None): maximum number of feature files to analyse. If None, analyse all feature
files.

    Returns:
        pd.DataFrame: training and validation feature-distribution statistics.
    """

    try:

        # store results for training and validation datasets
        distribution_results = []

        # process training and validation datasets
        for dataset_name, image_folder in [("Training", train_image_folder), ("Validation", valid_image_folder)]:

            # folder containing extracted feature files
            feature_folder = os.path.join(image_folder, "pt")

            # retrieve all saved feature files
            all_feature_files = sorted([
                filename
                for filename in os.listdir(feature_folder)
                if filename.lower().endswith(".pt")
            ])
```

```
        # optionally select files evenly across the dataset
        if (
            maximum_files is not None
            and len(all_feature_files) > maximum_files
        ):

            # generate evenly spaced indices across the complete feature-file list
            indices = torch.linspace(0, len(all_feature_files) - 1, steps=maximum_files).long()

            # select feature files using the evenly spaced indices
            feature_files = [
                all_feature_files[index]
                for index in indices
            ]

        else:

            # analyse all feature files
            feature_files = all_feature_files

        # initialise mean values
        mean_values = []

        # initialise standard deviations
        standard_deviations = []

        # initialise feature variances
        feature_variances = []

        # initialise feature norms
        feature_norms = []

        # process each feature file
        for filename in feature_files:

            # load saved visual feature file
            feature_data = torch.load(
                os.path.join(
                    feature_folder,
                    filename
                ),
```

```
                map_location="cpu",
                weights_only=True
            )

            # retrieve saved visual features and convert to floating-point values
            visual_features = (feature_data["visual_features"].float())

            # calculate mean values
            mean_values.append(visual_features.mean().item())

            # calculate standard deviations
            standard_deviations.append(visual_features.std(unbiased=False).item())

            # calculate feature variances
            feature_variances.append(visual_features.var(unbiased=False).item())

            # calculate feature norms
            feature_norms.append(visual_features.norm(dim=1).mean().item())

        # store results for current dataset
        distribution_results.append({

            "Dataset":
                dataset_name,

            "Images Sampled":
                len(feature_files),

            "Mean Activation":
                sum(mean_values) / len(mean_values),

            "Mean Standard Deviation":
                sum(standard_deviations)
                / len(standard_deviations),

            "Mean Variance":
                sum(feature_variances)
                / len(feature_variances),

            "Mean Region Norm":
                sum(feature_norms)
                / len(feature_norms)
```

```
        })

    # convert results to DataFrame
    feature_distribution = pd.DataFrame(distribution_results)

    # metrics to plot
    feature_metrics = ["Mean Activation", "Mean Standard Deviation", "Mean Variance", "Mean Region Norm"]

    # use dataset as DataFrame index for plotting
    comparison = feature_distribution.set_index("Dataset")

    # create comparison plots
    fig, axes = plt.subplots(1, 4, figsize=(20, 5))

    # plot each diagnostic
    for axis, metric in zip(axes, feature_metrics):

        # plot current metric as a bar chart for Training and Validation
        comparison[metric].plot(kind="bar", ax=axis)

        # set title
        axis.set_title(metric)

        # label y-axis
        axis.set_ylabel("Value")

        # label x-axis
        axis.set_xlabel("")

        # align Training and Validation x-axis labels horizontally
        axis.tick_params(axis="x", rotation=0)

    # set overall title
    fig.suptitle("Training vs Validation Feature Distribution")

    # automatically adjust spacing between subplots to prevent labels and titles from overlapping
    plt.tight_layout()

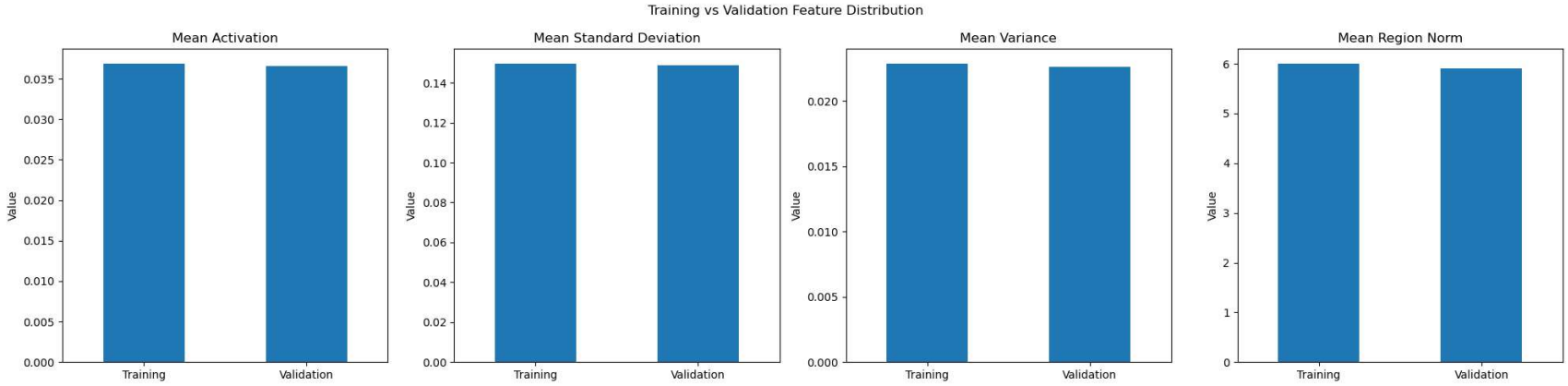
    # display plot
    plt.show()

    # return numerical results
```

```
        return feature_distribution

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [14]:
```
# calculate and plot Training versus Validation feature distribution statistics
feature_distribution = analyse_feature_distribution(train_image_folder, valid_image_folder)
```



In [15]:
```
# display as a table
display(feature_distribution)
```

	Dataset	Images Sampled	Mean Activation	Mean Standard Deviation	Mean Variance	Mean Region Norm
0	Training	59958	0.036838	0.149498	0.022848	5.998748
1	Validation	9904	0.036571	0.148846	0.022642	5.910683

Feature : a number produced by the ResNet-50 encoder to represent an edge, shape, texture or other visual pattern detected in an image.

- **Mean Activation:** i.e., the average of all features produced ror an image. Values for training (0.036838) and validation (0.036571) are almost identical, suggesting that the encoder is producing features at a very similar overall level for both datasets.

- **Mean Standard Deviation:** i.e., measures how much feature values differ from the average value. Training (0.149498) and validation (0.148846) values are almost identical, showing a very similar amount of feature variation in both datasets.

- **Mean Variance:** Similarly training (0.022848) and validation (0.022642) values are closely aligned indicating that the feature values have a consistent spread across both datasets

- **Mean Region Norm:** The encoder does not represent the whole image with only one set of numbers. It maintains the image as 49 spatial regions with each region represented by 2048 feature values. The region norm combines the 2048 values for a region into one number that represents the overall size of that region's feature vector. Training (5.998748) and validation (5.910683) values are again aligned

In [16]:
```
def calculate_feature_similarity(image_folder, dataset_name, sample_size=500):

    """
    Calculate and plot inter-image and within-image region similarity statistics

    Args:
        image_folder (str): folder containing the saved image feature files
        dataset_name (str): dataset name used in the plot titles.
        sample_size (int): maximum number of images to analyse, defaults to 500 due to memory constraints

    Returns:
        tuple: dictionaries containing inter-image and region similarity statistics.
    """

    try:

        # folder containing extracted feature files
        feature_folder = os.path.join(image_folder, "pt")

        # retrieve all saved feature files
        all_feature_files = sorted([filename for filename in os.listdir(feature_folder) if
filename.lower().endswith(".pt")])

        # select files evenly across the dataset when more than 500 feature files are available
        if len(all_feature_files) > sample_size:
```

```
            # generate evenly spaced indices across the complete dataset
            indices = torch.linspace(0, len(all_feature_files) - 1, steps=sample_size).long()

            # select the feature files using the evenly spaced indices
            feature_files = [all_feature_files[index] for index in indices]

        else:

            # use all feature files when the dataset contains fewer than 500 images
            feature_files = all_feature_files

        # store one feature representation for each image
        image_features = []

        # store mean within-image region similarity for each image
        image_region_similarities = []

        # process each selected feature file
        for filename in feature_files:

            # load extracted visual features
            feature_data = torch.load(os.path.join(feature_folder, filename), map_location="cpu",
weights_only=True)

            # retrieve visual feature tensor
            visual_features = feature_data["visual_features"].float()

            # average spatial regions to create one representation per image
            image_features.append(visual_features.mean(dim=0))

            # normalise each spatial region for cosine similarity
            normalised_regions = visual_features / visual_features.norm(dim=1, keepdim=True).clamp_min(1e-8)

            # calculate cosine similarity between all regions within the image
            region_similarity_matrix = normalised_regions @ normalised_regions.T

            # exclude comparisons of each region with itself
            region_mask = ~torch.eye(region_similarity_matrix.size(0), dtype=torch.bool)

            # store mean similarity between regions for this image
            image_region_similarities.append(region_similarity_matrix[region_mask].mean().item())
```

```
        # combine image-level feature representations
        image_features = torch.stack(image_features)

        # normalise image-level representations for cosine similarity
        image_features = image_features / image_features.norm(dim=1, keepdim=True).clamp_min(1e-8)

        # calculate cosine similarity between all selected images
        image_similarity_matrix = image_features @ image_features.T

        # select each unique image pair once and exclude self-comparisons
        image_mask = torch.triu(torch.ones_like(image_similarity_matrix, dtype=torch.bool), diagonal=1)

        # extract unique inter-image similarity values
        inter_image_similarities = image_similarity_matrix[image_mask]

        # convert within-image region similarities to a tensor
        region_similarities = torch.tensor(image_region_similarities)

        # summary statistics for similarity between different images
        inter_image_results = {
            "Images Sampled": len(feature_files),
            "Mean Similarity": inter_image_similarities.mean().item(),
            "Standard Deviation": inter_image_similarities.std().item(),
            "Minimum Similarity": inter_image_similarities.min().item(),
            "95th Percentile": torch.quantile(inter_image_similarities, 0.95).item(),
            "Maximum Similarity": inter_image_similarities.max().item()
        }

        # summary statistics for similarity between regions within images
        region_results = {
            "Images Sampled": len(feature_files),
            "Mean Region Similarity": region_similarities.mean().item(),
            "Standard Deviation": region_similarities.std().item(),
            "Minimum Similarity": region_similarities.min().item(),
            "95th Percentile": torch.quantile(region_similarities, 0.95).item(),
            "Maximum Similarity": region_similarities.max().item()
        }

        # exclude image count from inter-image similarity plot
        inter_image_metrics = {key: value for key, value in inter_image_results.items() if key != "Images Sampled"}

        # exclude image count from within-image region similarity plot
```

```
        region_metrics = {key: value for key, value in region_results.items() if key != "Images Sampled"}

        # create two side-by-side bar charts
        fig, axes = plt.subplots(1, 2, figsize=(16, 6))

        # plot inter-image similarity statistics
        axes[0].bar(inter_image_metrics.keys(), inter_image_metrics.values())

        # set inter-image chart title
        axes[0].set_title(f"{dataset_name} Inter-Image Feature Similarity")

        # set inter-image y-axis label
        axes[0].set_ylabel("Cosine Similarity")

        # rotate inter-image x-axis labels
        axes[0].tick_params(axis="x", rotation=30)

        # plot within-image region similarity statistics
        axes[1].bar(region_metrics.keys(), region_metrics.values())

        # set region chart title
        axes[1].set_title(f"{dataset_name} Within-Image Region Similarity")

        # set region y-axis label
        axes[1].set_ylabel("Cosine Similarity")

        # rotate region x-axis labels
        axes[1].tick_params(axis="x", rotation=30)

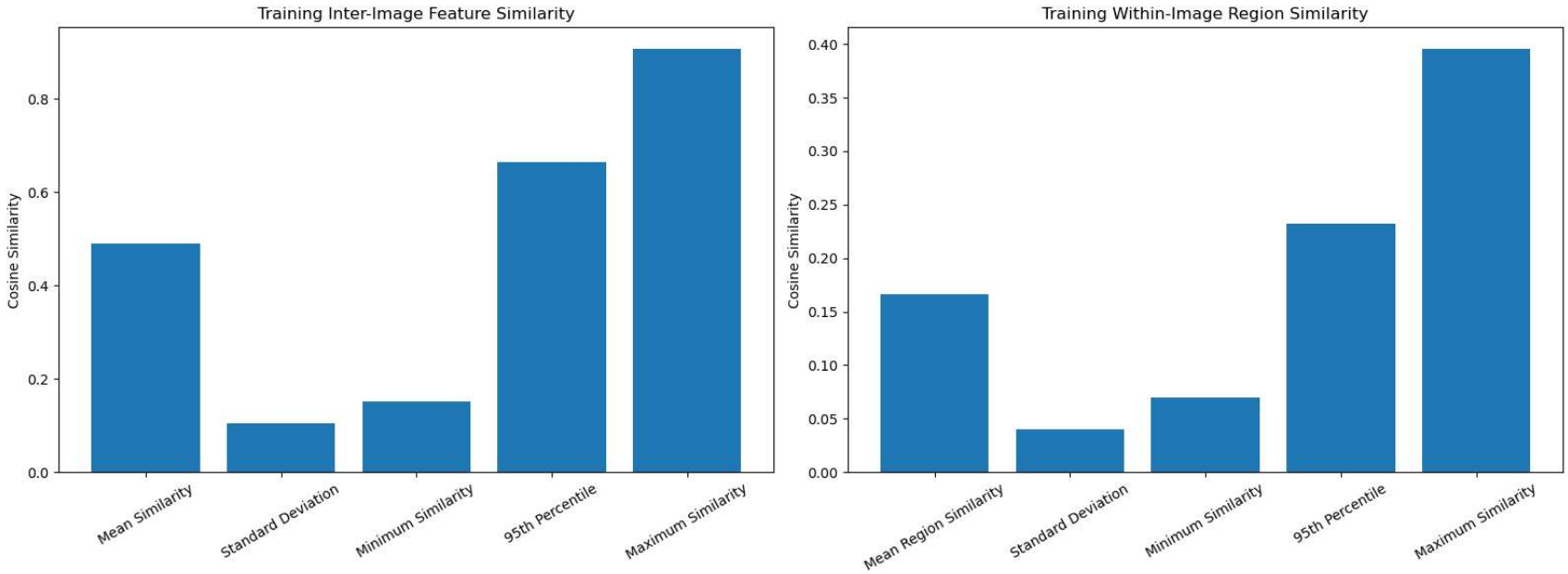
        # optimise plot spacing
        plt.tight_layout()

        # display plots
        plt.show()

        # return similarity statistics
        return inter_image_results, region_results

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [17]:
```
# calculate training similarity diagnostics
training_inter_image_similarity, training_region_similarity = calculate_feature_similarity(train_image_folder,
"Training")
```



In [18]:
```
# convert training inter-image similarity results to a table
training_inter_image_similarity_table = pd.DataFrame([training_inter_image_similarity])
```

In [19]:
```
# display table
training_inter_image_similarity_table
```

Out[19]:

	Images Sampled	Mean Similarity	Standard Deviation	Minimum Similarity	95th Percentile	Maximum Similarity
0	500	0.489284	0.104858	0.150715	0.664645	0.907927

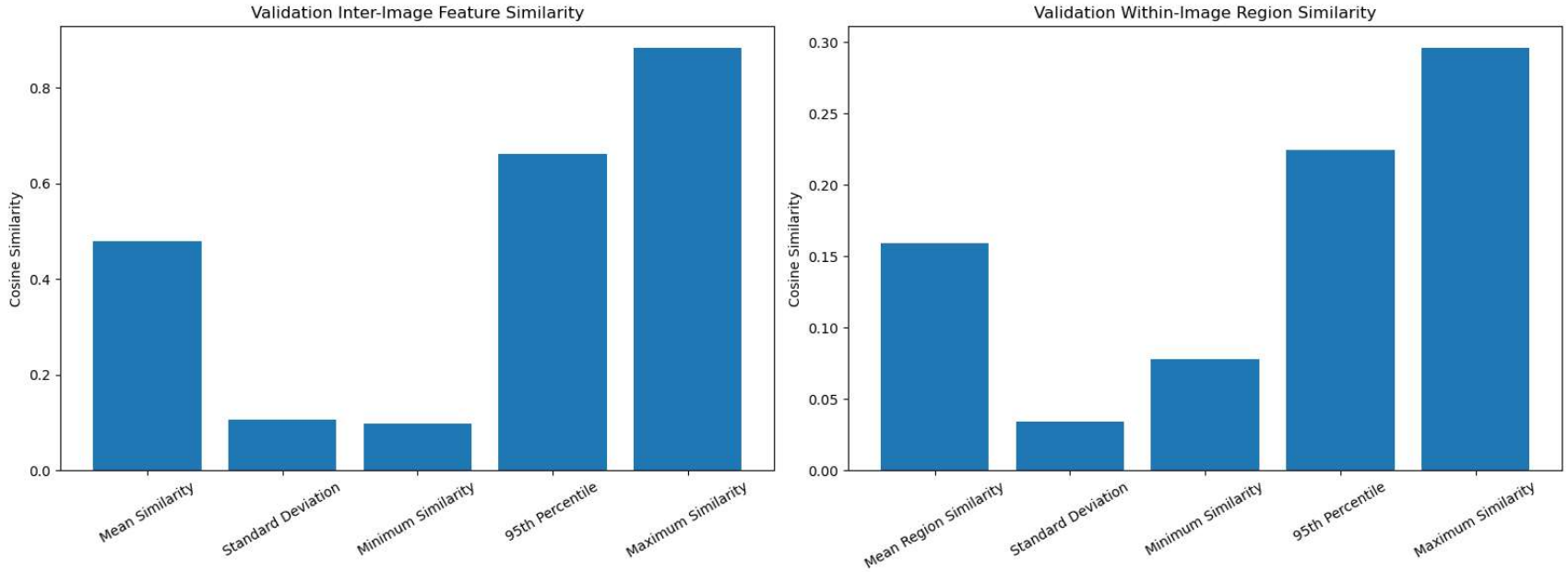
In [20]:
```
# convert training region similarity results to a table
training_region_similarity_table = pd.DataFrame([training_region_similarity])
```

In [21]:
```
# display table
training_region_similarity_table
```

Out[21]:

	Images Sampled	Mean Region Similarity	Standard Deviation	Minimum Similarity	95th Percentile	Maximum Similarity
0	500	0.166709	0.039657	0.069956	0.232005	0.395957

In [22]:
```
# calculate validation similarity diagnostics
validation_inter_image_similarity, validation_region_similarity = calculate_feature_similarity(valid_image_folder,
"Validation")
```



In [23]:
```
# convert validation inter-image similarity results to a table
validation_inter_image_similarity_table = pd.DataFrame([validation_inter_image_similarity])
```

In [24]:
```
# display table
validation_inter_image_similarity_table
```

Out[24]:

	Images Sampled	Mean Similarity	Standard Deviation	Minimum Similarity	95th Percentile	Maximum Similarity
0	500	0.479405	0.105799	0.09847	0.661625	0.884164

In [25]:
```
# convert validation region similarity results to a table
```

```
validation_region_similarity_table = pd.DataFrame([validation_region_similarity])
```

In [26]:
```
# display table
validation_region_similarity_table
```

Out[26]:

	Images Sampled	Mean Region Similarity	Standard Deviation	Minimum Similarity	95th Percentile	Maximum Similarity
0	500	0.159388	0.034249	0.07787	0.224073	0.296093

Cosine Similarity: measures how similar two sets of extracted features are. A value closer to 1 means the feature representations are more similar, while a value closer to 0 means they are less similar.

Inter-image Similarity compares the overall feature representation of one image with another image, i.e., similarity 'between images'. 500 images were selected from each of the training and validation datasets due to memory constraints. Every unique image pair within each sample was compared, giving 124,750 pairs per dataset. The datasets were analysed separately.

- **Mean Similarity:** i.e., the average similarity between all pairs of sampled images. The training value of 0.489284 and validation value of 0.479405 indicate a moderate level of similarity overall between images in their respective datasets. This is expected as medical images can share common visual structures while containing different content. The close values suggest that the overall level of inter-image similarity is consistent across the training and validation datasets.

- **Standard Deviation:** i.e., how much the similarity values vary around the average similarity. The training value of 0.104858 and validation value of 0.105799 are almost identical. This shows that the amount of variation between different image pairs is very similar in both datasets.

- **Minimum Similarity:** i.e., the lowest similarity found between any pair of sampled images. The training value is 0.150715 compared with 0.098470 for validation. The lower validation value indicates that the validation sample contains at least one pair of images with a greater difference in their feature representations. As this is an extreme value from a single pair, it does not by itself indicate a problem with the comparability of the datasets.

- **95th Percentile:** i.e., the similarity value that 95% of image pairs fall below. The training value is 0.664645 and the validation value is 0.661625. These values are again almost identical, indicating that 95% of image pairs in both datasets have a similarity below approximately 0.66. This suggests that the upper end of the similarity distribution is very consistent between training and validation sets.

- **Maximum Similarity:** i.e., the highest similarity found between any pair of sampled images. The training value is 0.907927 compared with 0.884164 for validation. Both values show that an image pair in each dataset has highly similar feature representations although this does not prove that they are duplicates.

Within-image Region Similarity compares the feature representations of different spatial regions within the same image. Each image is represented by 49 spatial regions, with each region containing 2,048 feature values. 500 images were selected from each of the training and validation datasets due to memory constraints. The datasets were analysed separately.

- **Mean Region Similarity:** i.e., the average similarity between the regions within each sampled image. The training value of 0.166709 and validation value of 0.159388 are very close and both are relatively low. This suggests that different regions within an image generally contain different feature information rather than repeatedly representing the same information. The values indicate similar regional behaviour across both datasets.

- **Standard Deviation:** i.e., how much the average region similarity varies between the sampled images. The training value is 0.039657 compared with 0.034249 for validation. Both values are low and relatively close, indicating that region similarity is fairly consistent across images in both datasets. The slightly lower validation value suggests slightly less variation in region similarity across validation images.

- **Minimum Similarity:** i.e., the lowest average region similarity found among the sampled images. The training value is 0.069956 compared with 0.077870 for validation. These values are close and indicate that, in both datasets, the image with the lowest average region similarity contains regions with greater differences in their feature representations.

- **95th Percentile:** i.e., the average region similarity value that 95% of the sampled images fall below. The training value is 0.232005 and the validation value is 0.224073. These values are very similar, meaning that 95% of sampled images have average region similarities below approximately 0.23 in both datasets. This suggests that the upper end of the region similarity distribution is consistent between training and validation.

- **Maximum Similarity:** i.e., the highest average region similarity found among the sampled images. The training value is 0.395957 compared with 0.296093 for validation. This shows that the training sample contains at least one image whose regions are more similar to one another than the most similar regions found in the validation sample. However, both values

remain well below 1, indicating that the regions are still producing different feature information rather than identical representations.

Overall: The training and validation datasets show consistent feature-similarity patterns. Their mean inter-image similarity, standard deviation and 95th percentile values are closely aligned, indicating that images in both datasets have a comparable overall level and spread of similarity. The within-image region similarity results also suggest that the encoder represents different image regions in a consistent way across both datasets. Although there are some differences in the minimum and maximum values, these represent extreme cases. Overall, the results suggest that the training and validation sets have a similar baseline in terms of the visual features extracted by the encoder.

Issues That Could Increase The Risk Of Hallucinations

- **Frozen encoder:** At this feature-extraction stage, the ResNet-50 encoder is frozen by default. At this stage its visual features are therefore based on RadImageNet pretraining rather than being adapted specifically to the ROCOv2 captioning task. If important image details are not represented effectively, the later captioning model may rely more heavily on learned language patterns and potentially generate unsupported information. This will be potentially addressed during fine-tuning

- **Loss of image detail during resizing:** Every image is resized to 224 × 224 pixels. Large medical images may therefore lose small or subtle visual details. If an important finding becomes difficult for the encoder to detect, a later captioning model may incorrectly infer or omit information.

- **Image shape is forced to 224 × 224:** The resize operation forces every image into a square rather than preserving its original aspect ratio. This can distort anatomical structures and could reduce the accuracy of the extracted features.

- **Limited spatial resolution:** Each image is ultimately represented by only 49 spatial regions, i.e., a 7 × 7 feature map. Small abnormalities may therefore be represented within the same region as nearby anatomical structures. This could make precise localisation more difficult for a later captioning model.

- **No explicit feature-shape check:** The pipeline does not explicitly check that every saved image feature has the expected shape of 49 × 2,048. An incorrectly shaped feature could therefore reach later stages without being identified immediately.

- **No explicit NaN or infinity check:** There is no explicit check that all extracted feature values are valid finite numbers. Invalid or corrupted feature values could affect the later captioning model and potentially produce unreliable output.

- **Zero or near-zero feature vectors are not explicitly reported:** The similarity analysis prevents division by zero, but it does not report how many image regions have zero or unusually small feature vectors. Poorly represented regions could therefore be hidden within the overall statistics.

- **Feature similarity does not measure semantic correctness:** Two images having similar encoder features does not prove that their medical findings or correct captions are the same. The similarity diagnostics test the behaviour of the visual representations, but they do not prove that those representations contain all of the information required to generate a factually correct caption.

References

BMEII-AI (n.d.) Pytorch_Example.ipynb, RadImageNet, GitHub.
Available At: https://github.com/BMEII-AI/RadImageNet/blob/main/pytorch_example.ipynb

Mei, X.,et al, 'RadImageNet: An Open Radiologic Deep Learning Research Dataset For Effective Transfer Learning', Radiology: Artificial Intelligence (2022). Available At : https://pubmed.ncbi.nlm.nih.gov/36204533/

OCOM5100M Programming For Data Science

OCOM5101M Data Science

OCOM5200M Machine Learning

OCOM5203M Deep Learning

OCOM5203M Deep Learning Module, Summative Assessment 2

OCOM5204M Data Mining And Analytics

Project Jupyter, Jupyter Notebook documentation. Available At: https://jupyter-notebook.readthedocs.io/en/stable/

Python Software Foundation, Python 3.13 documentation (2026). Available At: https://docs.python.org/3.13/

RadImageNet Official Code And Document Repository https://github.com/BMEII-AI/RadImageNet

RadImageNet_pytorch.zip for ResNet50.pt learned parameter tensors
https://drive.google.com/file/d/1RHt2GnuOYIc_gcoTETtBDSW73mFyRAtR/view

ROCOv2: Radiology Objects in Context Version 2, An Updated Multimodal Image Dataset https://zenodo.org/records/10821435

Ruckert, J.et al, 'ROCOv2: Radiology Objects In COntext Version 2, An Updated Multimodal Image Dataset' (2024).
Available At : https://pubmed.ncbi.nlm.nih.gov/38926396/